DATABASE TOOLS & OPERATIONS GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Audience**: Developers, DBAs, Data Engineers

---

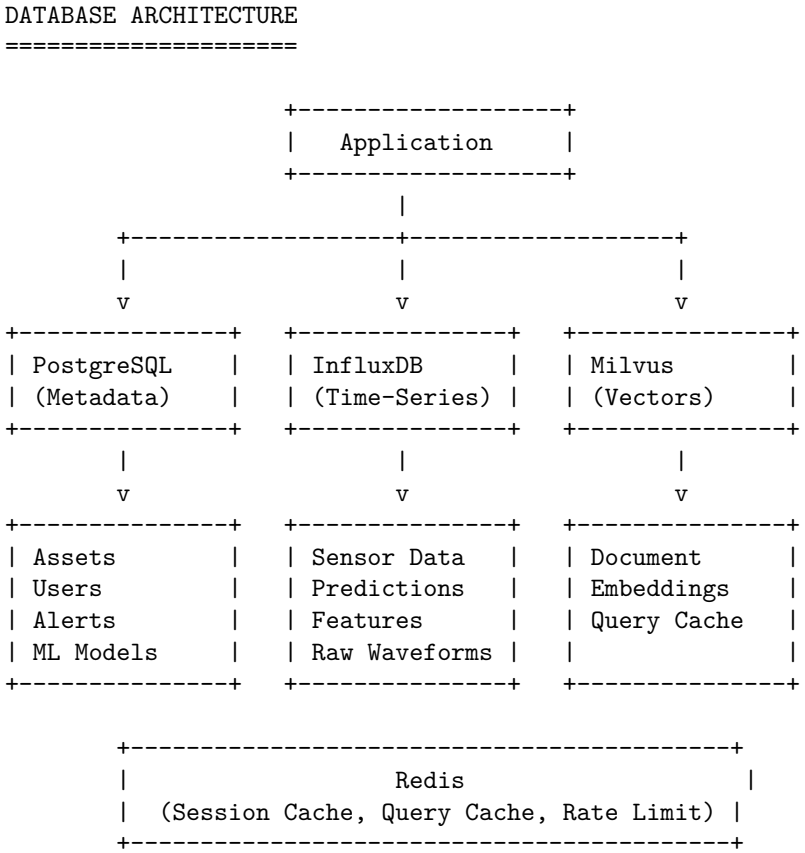
TABLE OF CONTENTS

1. Database Architecture Overview
2. PostgreSQL Operations
3. InfluxDB Operations
4. Milvus Vector Database
5. Redis Cache Operations
6. Database Migration Tools
7. Query Optimization
8. Backup & Recovery
9. Monitoring & Health Checks
10. Complete Operation Workflows

---

1. DATABASE ARCHITECTURE OVERVIEW

1.1 Database Stack



1.2 Connection Configuration

```
# ~/.database-connections
# Source this file: source ~/.database-connections

# PostgreSQL
export POSTGRES_HOST="postgres.oilgas.internal"
export POSTGRES_PORT="5432"
export POSTGRES_DB="oilgas_ai"
export POSTGRES_USER="app_user"
export POSTGRES_PASSWORD="secure-password"
export DATABASE_URL="postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@${POSTGRES_HOST}:${POSTGRES_POR

# InfluxDB
export INFLUXDB_URL="http://influxdb.oilgas.internal:8086"
export INFLUXDB_ORG="oilgas"
export INFLUXDB_TOKEN="your-influxdb-token"
export INFLUXDB_BUCKET="sensor_data"

# Milvus
export MILVUS_HOST="milvus.oilgas.internal"
export MILVUS_PORT="19530"
export MILVUS_URI="http://${MILVUS_HOST}:${MILVUS_PORT}"

# Redis
export REDIS_HOST="redis.oilgas.internal"
export REDIS_PORT="6379"
export REDIS_PASSWORD="redis-password"
export REDIS_URL="redis://:${REDIS_PASSWORD}@${REDIS_HOST}:${REDIS_PORT}/0"
```

---

2. POSTGRESQL OPERATIONS

2.1 Installation & Client Setup

```
# Install PostgreSQL 16 client
sudo apt-get update
sudo apt-get install -y postgresql-client-16

# Install enhanced CLI tools
pip install pgcli  # Interactive CLI with autocomplete

# Verify installation
psql --version
pgcli --version
```

2.2 Connection Commands

```
# Connect using environment variables
psql "postgresql://$POSTGRES_USER:$POSTGRES_PASSWORD@$POSTGRES_HOST:$POSTGRES_PORT/$POSTGRES_DB"

# Connect with parameters
psql -h $POSTGRES_HOST -p $POSTGRES_PORT -U $POSTGRES_USER -d $POSTGRES_DB
```

```bash
# Connect using pgcli (enhanced)
pgcli -h $POSTGRES_HOST -p $POSTGRES_PORT -U $POSTGRES_USER -d $POSTGRES_DB

# Connect with SSL
psql "postgresql://$POSTGRES_USER:$POSTGRES_PASSWORD@$POSTGRES_HOST:$POSTGRES_PORT/$POSTGRES_DB?sslmode=

# Using .pgpass file (no password prompt)
echo "$POSTGRES_HOST:$POSTGRES_PORT:$POSTGRES_DB:$POSTGRES_USER:$POSTGRES_PASSWORD" >> ~/.pgpass
chmod 600 ~/.pgpass
```

2.3 Database Navigation

```bash
-- List all databases
\l
\l+  -- with sizes

-- Switch database
\c oilgas_ai

-- List schemas
\dn
\dn+  -- with privileges

-- List tables in schema
\dt asset_mgmt.*
\dt+  -- with sizes

-- Describe table
\d asset_mgmt.assets
\d+ asset_mgmt.assets  -- detailed

-- List indexes
\di asset_mgmt.*

-- List views
\dv asset_mgmt.*

-- List functions
\df

-- Show table privileges
\dp asset_mgmt.assets

-- Show current connection info
\conninfo

-- Show search path
SHOW search_path;
```

2.4 CRUD Operations

```bash
-- ========================================
-- CREATE Operations
-- ========================================
```

```sql
-- Insert single row
INSERT INTO asset_mgmt.assets (
    asset_tag, name, asset_type, status, criticality
) VALUES (
    'PUMP-NEW-001', 'New Pump Unit', 'pump', 'operational', 'high'
) RETURNING asset_id;

-- Insert multiple rows
INSERT INTO asset_mgmt.sensors (
    sensor_tag, asset_id, sensor_type, unit, min_value, max_value
) VALUES
    ('TEMP-001', 1, 'temperature', 'celsius', -20, 150),
    ('VIB-001', 1, 'vibration', 'mm/s', 0, 50),
    ('PRES-001', 1, 'pressure', 'bar', 0, 100)
RETURNING sensor_id;

-- Insert with ON CONFLICT (upsert)
INSERT INTO asset_mgmt.assets (asset_tag, name, status)
VALUES ('PUMP-001', 'Pump Unit 1', 'operational')
ON CONFLICT (asset_tag) DO UPDATE
SET status = EXCLUDED.status,
    updated_at = NOW();

-- ======================================
-- READ Operations
-- ======================================

-- Select with conditions
SELECT
    a.asset_id,
    a.asset_tag,
    a.name,
    a.status,
    COUNT(s.sensor_id) as sensor_count
FROM asset_mgmt.assets a
LEFT JOIN asset_mgmt.sensors s ON a.asset_id = s.asset_id
WHERE a.status = 'operational'
GROUP BY a.asset_id
ORDER BY a.asset_tag;

-- Select with JSON operations
SELECT
    asset_id,
    metadata->>'manufacturer' as manufacturer,
    metadata->>'model' as model,
    metadata->'specs'->>'power_rating' as power_rating
FROM asset_mgmt.assets
WHERE metadata->>'manufacturer' = 'Emerson';

-- Window functions
SELECT
    a.asset_tag,
    p.prediction_time,
```

```
    p.predicted_rul,
    AVG(p.predicted_rul) OVER (
        PARTITION BY a.asset_id
        ORDER BY p.prediction_time
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) as avg_rul_7day
FROM ml_models.prediction_log p
JOIN asset_mgmt.assets a ON p.asset_id = a.asset_id
WHERE p.prediction_type = 'rul'
ORDER BY a.asset_tag, p.prediction_time DESC;

-- CTE (Common Table Expression)
WITH recent_alerts AS (
    SELECT
        asset_id,
        COUNT(*) as alert_count,
        MAX(triggered_at) as last_alert
    FROM alerts.alert_instances
    WHERE triggered_at > NOW() - INTERVAL '7 days'
    GROUP BY asset_id
)
SELECT
    a.asset_tag,
    a.name,
    COALESCE(ra.alert_count, 0) as alerts_7d,
    ra.last_alert
FROM asset_mgmt.assets a
LEFT JOIN recent_alerts ra ON a.asset_id = ra.asset_id
ORDER BY alerts_7d DESC;

-- ======================================
-- UPDATE Operations
-- ======================================

-- Simple update
UPDATE asset_mgmt.assets
SET status = 'maintenance',
    updated_at = NOW()
WHERE asset_id = 1
RETURNING *;

-- Update with JOIN
UPDATE asset_mgmt.sensors s
SET status = 'inactive'
FROM asset_mgmt.assets a
WHERE s.asset_id = a.asset_id
AND a.status = 'decommissioned';

-- Update JSON field
UPDATE asset_mgmt.assets
SET metadata = jsonb_set(
    metadata,
    '{last_maintenance}',
    to_jsonb(NOW()::text)
```

```
)
WHERE asset_id = 1;

-- ======================================
-- DELETE Operations
-- ======================================

-- Delete with condition
DELETE FROM alerts.alert_instances
WHERE resolved_at < NOW() - INTERVAL '90 days';

-- Delete with CASCADE (if foreign keys allow)
DELETE FROM asset_mgmt.assets
WHERE asset_id = 999;

-- Truncate table (fast delete all)
TRUNCATE TABLE temp_import_data;
TRUNCATE TABLE temp_import_data RESTART IDENTITY CASCADE;
```

2.5 Advanced Queries

```
-- ======================================
-- Performance Statistics
-- ======================================

-- Table sizes
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) as total_size,
    pg_size_pretty(pg_relation_size(schemaname || '.' || tablename)) as table_size,
    pg_size_pretty(pg_indexes_size(schemaname || '.' || tablename)) as index_size
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;

-- Active queries
SELECT
    pid,
    usename,
    state,
    query_start,
    NOW() - query_start as duration,
    LEFT(query, 100) as query_preview
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start;

-- Index usage
SELECT
    schemaname,
    tablename,
    indexname,
    idx_tup_read,
```

```
    idx_tup_fetch,
    ROUND(100.0 * idx_tup_fetch / NULLIF(idx_tup_read, 0), 2) as fetch_ratio
FROM pg_stat_user_indexes
ORDER BY idx_tup_read DESC;

-- Cache hit ratio
SELECT
    sum(heap_blks_read) as heap_read,
    sum(heap_blks_hit) as heap_hit,
    ROUND(100.0 * sum(heap_blks_hit) / (sum(heap_blks_hit) + sum(heap_blks_read)), 2) as cache_hit_ratio
FROM pg_statio_user_tables;

-- ========================================
-- Explain & Analyze
-- ========================================

-- Explain query plan
EXPLAIN (FORMAT JSON)
SELECT * FROM asset_mgmt.assets WHERE status = 'operational';

-- Explain with actual execution
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT a.*, COUNT(s.sensor_id)
FROM asset_mgmt.assets a
LEFT JOIN asset_mgmt.sensors s ON a.asset_id = s.asset_id
WHERE a.status = 'operational'
GROUP BY a.asset_id;
```

2.6 Python PostgreSQL Client

```
#!/usr/bin/env python3
# scripts/postgres_client.py
"""
PostgreSQL Database Client

Usage:
    from postgres_client import PostgresClient
    db = PostgresClient()
    assets = db.get_assets()
"""

import os
from contextlib import contextmanager
from typing import List, Dict, Any, Optional
import pandas as pd
from sqlalchemy import create_engine, text, MetaData
from sqlalchemy.orm import sessionmaker
from loguru import logger


class PostgresClient:
    """PostgreSQL database operations"""

    def __init__(self, database_url: str = None):
```

```
        self.database_url = database_url or os.environ.get('DATABASE_URL')

        if not self.database_url:
            host = os.environ.get('POSTGRES_HOST', 'localhost')
            port = os.environ.get('POSTGRES_PORT', '5432')
            db = os.environ.get('POSTGRES_DB', 'oilgas_ai')
            user = os.environ.get('POSTGRES_USER', 'app_user')
            password = os.environ.get('POSTGRES_PASSWORD', '')
            self.database_url = f"postgresql://{user}:{password}@{host}:{port}/{db}"

        self.engine = create_engine(
            self.database_url,
            pool_size=10,
            max_overflow=20,
            pool_pre_ping=True
        )
        self.Session = sessionmaker(bind=self.engine)
        self.metadata = MetaData()

    @contextmanager
    def get_session(self):
        """Get database session with automatic cleanup"""
        session = self.Session()
        try:
            yield session
            session.commit()
        except Exception:
            session.rollback()
            raise
        finally:
            session.close()

    def execute_query(self, query: str, params: dict = None) -> pd.DataFrame:
        """Execute SQL query and return DataFrame"""
        with self.engine.connect() as conn:
            result = pd.read_sql(text(query), conn, params=params)
        return result

    def execute_statement(self, statement: str, params: dict = None) -> int:
        """Execute SQL statement (INSERT/UPDATE/DELETE)"""
        with self.engine.connect() as conn:
            result = conn.execute(text(statement), params or {})
            conn.commit()
            return result.rowcount

    # ========================================
    # Asset Management Operations
    # ========================================

    def get_assets(
        self,
        status: str = None,
        asset_type: str = None,
        criticality: str = None
```

```
) -> pd.DataFrame:
    """Get assets with optional filters"""
    query = """
    SELECT
        a.asset_id,
        a.asset_tag,
        a.name,
        a.asset_type,
        a.status,
        a.criticality,
        l.name as location_name,
        a.metadata,
        a.created_at,
        a.updated_at
    FROM asset_mgmt.assets a
    LEFT JOIN asset_mgmt.locations l ON a.location_id = l.location_id
    WHERE 1=1
    """
    params = {}

    if status:
        query += " AND a.status = :status"
        params['status'] = status
    if asset_type:
        query += " AND a.asset_type = :asset_type"
        params['asset_type'] = asset_type
    if criticality:
        query += " AND a.criticality = :criticality"
        params['criticality'] = criticality

    query += " ORDER BY a.asset_tag"

    return self.execute_query(query, params)

def get_asset_by_id(self, asset_id: int) -> Optional[Dict]:
    """Get single asset by ID"""
    query = """
    SELECT * FROM asset_mgmt.assets WHERE asset_id = :asset_id
    """
    df = self.execute_query(query, {'asset_id': asset_id})
    if df.empty:
        return None
    return df.iloc[0].to_dict()

def get_asset_by_tag(self, asset_tag: str) -> Optional[Dict]:
    """Get single asset by tag"""
    query = """
    SELECT * FROM asset_mgmt.assets WHERE asset_tag = :asset_tag
    """
    df = self.execute_query(query, {'asset_tag': asset_tag})
    if df.empty:
        return None
    return df.iloc[0].to_dict()
```

```
def create_asset(self, asset_data: Dict) -> int:
    """Create new asset"""
    statement = """
    INSERT INTO asset_mgmt.assets (
        asset_tag, name, asset_type, status, criticality,
        location_id, metadata
    ) VALUES (
        :asset_tag, :name, :asset_type, :status, :criticality,
        :location_id, :metadata::jsonb
    )
    RETURNING asset_id
    """

    with self.engine.connect() as conn:
        result = conn.execute(text(statement), asset_data)
        conn.commit()
        return result.fetchone()[0]

def update_asset(self, asset_id: int, updates: Dict) -> bool:
    """Update asset"""
    set_clauses = []
    params = {'asset_id': asset_id}

    for key, value in updates.items():
        set_clauses.append(f"{key} = :{key}")
        params[key] = value

    set_clauses.append("updated_at = NOW()")

    statement = f"""
    UPDATE asset_mgmt.assets
    SET {', '.join(set_clauses)}
    WHERE asset_id = :asset_id
    """

    return self.execute_statement(statement, params) > 0

def delete_asset(self, asset_id: int) -> bool:
    """Delete asset"""
    statement = "DELETE FROM asset_mgmt.assets WHERE asset_id = :asset_id"
    return self.execute_statement(statement, {'asset_id': asset_id}) > 0

# ========================================
# Sensor Operations
# ========================================

def get_sensors_by_asset(self, asset_id: int) -> pd.DataFrame:
    """Get all sensors for an asset"""
    query = """
    SELECT * FROM asset_mgmt.sensors
    WHERE asset_id = :asset_id
    ORDER BY sensor_tag
    """
    return self.execute_query(query, {'asset_id': asset_id})
```

```
def create_sensor(self, sensor_data: Dict) -> int:
    """Create new sensor"""
    statement = """
    INSERT INTO asset_mgmt.sensors (
        sensor_tag, asset_id, sensor_type, unit,
        min_value, max_value, status
    ) VALUES (
        :sensor_tag, :asset_id, :sensor_type, :unit,
        :min_value, :max_value, :status
    )
    RETURNING sensor_id
    """

    with self.engine.connect() as conn:
        result = conn.execute(text(statement), sensor_data)
        conn.commit()
        return result.fetchone()[0]

# ========================================
# Alert Operations
# ========================================

def get_active_alerts(self, limit: int = 100) -> pd.DataFrame:
    """Get active alerts"""
    query = """
    SELECT
        ai.alert_id,
        a.asset_tag,
        a.name as asset_name,
        ad.alert_name,
        ad.severity,
        ai.triggered_value,
        ai.triggered_at,
        ai.status
    FROM alerts.alert_instances ai
    JOIN alerts.alert_definitions ad ON ai.definition_id = ad.definition_id
    JOIN asset_mgmt.assets a ON ai.asset_id = a.asset_id
    WHERE ai.status = 'active'
    ORDER BY ai.triggered_at DESC
    LIMIT :limit
    """
    return self.execute_query(query, {'limit': limit})

def acknowledge_alert(self, alert_id: int, user_id: int, notes: str = None) -> bool:
    """Acknowledge an alert"""
    statement = """
    UPDATE alerts.alert_instances
    SET status = 'acknowledged',
        acknowledged_at = NOW(),
        acknowledged_by = :user_id,
        notes = :notes
    WHERE alert_id = :alert_id AND status = 'active'
    """
```

```
        return self.execute_statement(statement, {
            'alert_id': alert_id,
            'user_id': user_id,
            'notes': notes
        }) > 0

    def resolve_alert(self, alert_id: int, resolution_notes: str = None) -> bool:
        """Resolve an alert"""
        statement = """
        UPDATE alerts.alert_instances
        SET status = 'resolved',
            resolved_at = NOW(),
            notes = COALESCE(notes, '') || E'\nResolution: ' || :resolution_notes
        WHERE alert_id = :alert_id
        """
        return self.execute_statement(statement, {
            'alert_id': alert_id,
            'resolution_notes': resolution_notes or 'Resolved'
        }) > 0

    # ========================================
    # ML Model Operations
    # ========================================

    def get_model_registry(self, status: str = 'active') -> pd.DataFrame:
        """Get registered ML models"""
        query = """
        SELECT
            m.model_id,
            m.model_name,
            m.model_version,
            m.model_type,
            m.framework,
            m.metrics,
            m.created_at,
            d.deployment_target,
            d.deployed_at,
            d.status as deployment_status
        FROM ml_models.model_registry m
        LEFT JOIN ml_models.model_deployments d ON m.model_id = d.model_id
        WHERE d.status = :status OR d.status IS NULL
        ORDER BY m.model_name, m.model_version DESC
        """
        return self.execute_query(query, {'status': status})

    def register_model(self, model_data: Dict) -> int:
        """Register new ML model"""
        statement = """
        INSERT INTO ml_models.model_registry (
            model_name, model_version, model_type, framework,
            model_path, metrics, training_config
        ) VALUES (
            :model_name, :model_version, :model_type, :framework,
            :model_path, :metrics::jsonb, :training_config::jsonb
```

```
        )
        RETURNING model_id
        """

        with self.engine.connect() as conn:
            result = conn.execute(text(statement), model_data)
            conn.commit()
            return result.fetchone()[0]

    def log_prediction(self, prediction_data: Dict) -> int:
        """Log ML prediction"""
        statement = """
        INSERT INTO ml_models.prediction_log (
            model_id, asset_id, prediction_type,
            input_features, predicted_value, confidence,
            prediction_time
        ) VALUES (
            :model_id, :asset_id, :prediction_type,
            :input_features::jsonb, :predicted_value, :confidence,
            :prediction_time
        )
        RETURNING prediction_id
        """

        with self.engine.connect() as conn:
            result = conn.execute(text(statement), prediction_data)
            conn.commit()
            return result.fetchone()[0]

    # ========================================
    # Knowledge Base Operations
    # ========================================

    def get_documents(self, status: str = None) -> pd.DataFrame:
        """Get indexed documents"""
        query = """
        SELECT
            d.document_id,
            d.file_name,
            d.file_type,
            d.file_size_bytes,
            d.indexed_at,
            d.status,
            COUNT(c.chunk_id) as chunk_count
        FROM knowledge.documents d
        LEFT JOIN knowledge.chunks c ON d.document_id = c.document_id
        """
        params = {}

        if status:
            query += " WHERE d.status = :status"
            params['status'] = status

        query += " GROUP BY d.document_id ORDER BY d.indexed_at DESC"
```

```
        return self.execute_query(query, params)

    def create_document(self, doc_data: Dict) -> int:
        """Create document record"""
        statement = """
        INSERT INTO knowledge.documents (
            file_name, file_path, file_type, file_size_bytes, status
        ) VALUES (
            :file_name, :file_path, :file_type, :file_size_bytes, 'pending'
        )
        RETURNING document_id
        """

        with self.engine.connect() as conn:
            result = conn.execute(text(statement), doc_data)
            conn.commit()
            return result.fetchone()[0]

    # ========================================
    # Health & Statistics
    # ========================================

    def get_table_stats(self) -> pd.DataFrame:
        """Get table statistics"""
        query = """
        SELECT
            schemaname as schema,
            tablename as table,
            n_live_tup as row_count,
            pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) as total_size,
            last_vacuum,
            last_analyze
        FROM pg_stat_user_tables
        ORDER BY n_live_tup DESC
        """
        return self.execute_query(query)

    def health_check(self) -> Dict:
        """Check database health"""
        try:
            # Check connection
            result = self.execute_query("SELECT 1 as connected")

            # Get active connections
            conn_query = """
            SELECT count(*) as active_connections
            FROM pg_stat_activity
            WHERE state = 'active'
            """
            connections = self.execute_query(conn_query)

            # Get database size
            size_query = """
```

```
            SELECT pg_size_pretty(pg_database_size(current_database())) as db_size
            """
            size = self.execute_query(size_query)

            return {
                'status': 'healthy',
                'connected': True,
                'active_connections': connections.iloc[0]['active_connections'],
                'database_size': size.iloc[0]['db_size']
            }
        except Exception as e:
            return {
                'status': 'unhealthy',
                'connected': False,
                'error': str(e)
            }


# CLI usage
if __name__ == '__main__':
    import argparse

    parser = argparse.ArgumentParser(description='PostgreSQL Client')
    parser.add_argument('command', choices=['health', 'assets', 'alerts', 'models', 'stats'])
    parser.add_argument('--status', help='Filter by status')
    parser.add_argument('--output', help='Output file')

    args = parser.parse_args()

    client = PostgresClient()

    if args.command == 'health':
        print(client.health_check())
    elif args.command == 'assets':
        df = client.get_assets(status=args.status)
        if args.output:
            df.to_csv(args.output, index=False)
        else:
            print(df.to_string())
    elif args.command == 'alerts':
        df = client.get_active_alerts()
        print(df.to_string())
    elif args.command == 'models':
        df = client.get_model_registry(status=args.status or 'active')
        print(df.to_string())
    elif args.command == 'stats':
        df = client.get_table_stats()
        print(df.to_string())
```

---

3. INFLUXDB OPERATIONS

3.1 Installation & Setup

```
# Install InfluxDB CLI
wget https://dl.influxdata.com/influxdb/releases/influxdb2-client-2.7.3-linux-amd64.tar.gz
tar xvzf influxdb2-client-2.7.3-linux-amd64.tar.gz
sudo mv influx /usr/local/bin/

# Configure CLI
influx config create \
  --config-name oilgas \
  --host-url $INFLUXDB_URL \
  --org $INFLUXDB_ORG \
  --token $INFLUXDB_TOKEN \
  --active

# Verify connection
influx ping
influx version
```

3.2 Bucket Operations

```
# List buckets
influx bucket list

# Create bucket
influx bucket create \
  --name sensor_data \
  --retention 365d \
  --description "Sensor readings data"

influx bucket create \
  --name predictions \
  --retention 180d \
  --description "ML prediction results"

influx bucket create \
  --name features \
  --retention 90d \
  --description "Computed features"

# Update bucket retention
influx bucket update \
  --id BUCKET_ID \
  --retention 730d

# Delete bucket
influx bucket delete --name old_bucket
```

3.3 Flux Query Language

```
# ======================================
# Basic Queries
# ======================================
```

```
# Query recent data
influx query '
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> limit(n: 100)
'

# Query specific asset
influx query '
from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> filter(fn: (r) => r.asset_id == "PUMP-001")
  |> filter(fn: (r) => r._field == "vibration")
'

# Multiple filters
influx query '
from(bucket: "sensor_data")
  |> range(start: -7d)
  |> filter(fn: (r) =>
      r._measurement == "sensor_readings" and
      r.asset_id == "PUMP-001" and
      (r._field == "temperature" or r._field == "vibration")
  )
'

# ========================================
# Aggregations
# ========================================

# Mean per hour
influx query '
from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)
  |> yield(name: "hourly_mean")
'

# Multiple aggregations
influx query '
from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> filter(fn: (r) => r._field == "vibration")
  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)
  |> duplicate(column: "_value", as: "mean")
  |> aggregateWindow(every: 1h, fn: max, createEmpty: false)
  |> duplicate(column: "_value", as: "max")
  |> aggregateWindow(every: 1h, fn: min, createEmpty: false)
  |> duplicate(column: "_value", as: "min")
```

```
'

# Percentiles
influx query '
from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> filter(fn: (r) => r._field == "vibration")
  |> aggregateWindow(
      every: 1h,
      fn: (tables=<-, column) => tables |> quantile(q: 0.95, column: column)
  )
'

# ======================================
# Transformations
# ======================================

# Pivot (fields to columns)
influx query '
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
'

# Group by
influx query '
from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> group(columns: ["asset_id"])
  |> count()
'

# Derivative (rate of change)
influx query '
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._field == "temperature")
  |> derivative(unit: 1m, nonNegative: false)
'

# Moving average
influx query '
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._field == "vibration")
  |> movingAverage(n: 10)
'

# ======================================
# Anomaly Detection
# ======================================
```

```
# Values outside 3 standard deviations
influx query '
from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._field == "vibration")
  |> aggregateWindow(every: 5m, fn: mean)
  |> stddev()
  |> map(fn: (r) => ({
      r with
      upper_bound: r._value + (r.stddev * 3.0),
      lower_bound: r._value - (r.stddev * 3.0)
  }))
'

# ========================================
# Export Results
# ========================================

# Export to CSV
influx query '
from(bucket: "sensor_data")
  |> range(start: -7d)
  |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
' --raw > sensor_data.csv
```

3.4 Write Operations

```
# Write single point (line protocol)
influx write \
  --bucket sensor_data \
  --precision ns \
  'sensor_readings,asset_id=PUMP-001,sensor_type=vibration value=4.5 1702000000000000000'

# Write from file
influx write \
  --bucket sensor_data \
  --file sensor_data.lp

# Write from CSV
influx write \
  --bucket sensor_data \
  --file sensor_data.csv \
  --header "#constant measurement,sensor_readings" \
  --header "#datatype dateTime:RFC3339,tag,tag,double,double,double"
```

3.5 Python InfluxDB Client

```
#!/usr/bin/env python3
# scripts/influxdb_client.py
"""
InfluxDB Database Client

Usage:
```

```
    from influxdb_client_wrapper import InfluxDBClient
    client = InfluxDBClient()
    data = client.query_sensor_data("PUMP-001", "-24h")
"""

import os
from datetime import datetime
from typing import List, Dict, Optional
import pandas as pd
from influxdb_client import InfluxDBClient as InfluxClient
from influxdb_client import Point, WriteOptions
from influxdb_client.client.write_api import SYNCHRONOUS
from loguru import logger


class InfluxDBClientWrapper:
    """InfluxDB operations wrapper"""

    def __init__(self):
        self.url = os.environ.get('INFLUXDB_URL', 'http://localhost:8086')
        self.token = os.environ.get('INFLUXDB_TOKEN')
        self.org = os.environ.get('INFLUXDB_ORG', 'oilgas')

        self.client = InfluxClient(
            url=self.url,
            token=self.token,
            org=self.org
        )
        self.query_api = self.client.query_api()
        self.write_api = self.client.write_api(write_options=SYNCHRONOUS)

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.close()

    def close(self):
        """Close client connection"""
        self.client.close()

    # ========================================
    # Query Operations
    # ========================================

    def query(self, flux_query: str) -> pd.DataFrame:
        """Execute Flux query and return DataFrame"""
        result = self.query_api.query_data_frame(flux_query)

        if isinstance(result, list):
            if len(result) == 0:
                return pd.DataFrame()
            result = pd.concat(result, ignore_index=True)
```

```
        return result

    def query_sensor_data(
        self,
        asset_id: str,
        start: str,
        end: str = "now()",
        fields: List[str] = None,
        bucket: str = "sensor_data"
    ) -> pd.DataFrame:
        """
        Query sensor data for an asset.

        Args:
            asset_id: Asset identifier
            start: Start time (ISO or relative like "-1h")
            end: End time
            fields: Specific fields to query (None = all)
            bucket: InfluxDB bucket name
        """
        query = f'''
        from(bucket: "{bucket}")
          |> range(start: {start}, stop: {end})
          |> filter(fn: (r) => r._measurement == "sensor_readings")
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
        '''

        if fields:
            field_filter = ' or '.join([f'r._field == "{f}"' for f in fields])
            query += f'  |> filter(fn: (r) => {field_filter})\n'

        query += '  |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")'

        return self.query(query)

    def query_vibration_features(
        self,
        asset_id: str,
        start: str,
        end: str = "now()"
    ) -> pd.DataFrame:
        """Query computed vibration features"""
        query = f'''
        from(bucket: "features")
          |> range(start: {start}, stop: {end})
          |> filter(fn: (r) => r._measurement == "vibration_features")
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
        '''
        return self.query(query)

    def query_predictions(
        self,
        asset_id: str,
```

```
    prediction_type: str,
    start: str,
    end: str = "now()"
) -> pd.DataFrame:
    """Query ML predictions"""
    query = f'''
    from(bucket: "predictions")
      |> range(start: {start}, stop: {end})
      |> filter(fn: (r) => r.asset_id == "{asset_id}")
      |> filter(fn: (r) => r.prediction_type == "{prediction_type}")
      |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
    '''
    return self.query(query)

def query_aggregated(
    self,
    asset_id: str,
    field: str,
    start: str,
    interval: str = "1h",
    agg_func: str = "mean",
    bucket: str = "sensor_data"
) -> pd.DataFrame:
    """Query aggregated sensor data"""
    query = f'''
    from(bucket: "{bucket}")
      |> range(start: {start})
      |> filter(fn: (r) => r._measurement == "sensor_readings")
      |> filter(fn: (r) => r.asset_id == "{asset_id}")
      |> filter(fn: (r) => r._field == "{field}")
      |> aggregateWindow(every: {interval}, fn: {agg_func}, createEmpty: false)
    '''
    return self.query(query)

def list_assets(self, bucket: str = "sensor_data") -> List[str]:
    """List all unique asset IDs"""
    query = f'''
    import "influxdata/influxdb/schema"
    schema.tagValues(bucket: "{bucket}", tag: "asset_id")
    '''
    result = self.query_api.query(query)
    return [record.get_value() for table in result for record in table.records]

def list_measurements(self, bucket: str = "sensor_data") -> List[str]:
    """List all measurements"""
    query = f'''
    import "influxdata/influxdb/schema"
    schema.measurements(bucket: "{bucket}")
    '''
    result = self.query_api.query(query)
    return [record.get_value() for table in result for record in table.records]

def get_data_range(self, asset_id: str, bucket: str = "sensor_data") -> Dict:
    """Get data time range for an asset"""
```

```
        first_query = f'''
        from(bucket: "{bucket}")
          |> range(start: -365d)
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> first()
        '''
        last_query = f'''
        from(bucket: "{bucket}")
          |> range(start: -365d)
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> last()
        '''

        first_result = self.query_api.query(first_query)
        last_result = self.query_api.query(last_query)

        first_time = first_result[0].records[0].get_time() if first_result else None
        last_time = last_result[0].records[0].get_time() if last_result else None

        return {
            'first': first_time,
            'last': last_time
        }

    # ========================================
    # Write Operations
    # ========================================

    def write_point(
        self,
        measurement: str,
        tags: Dict,
        fields: Dict,
        timestamp: datetime = None,
        bucket: str = "sensor_data"
    ):
        """Write single data point"""
        point = Point(measurement)

        for key, value in tags.items():
            point = point.tag(key, value)

        for key, value in fields.items():
            point = point.field(key, value)

        if timestamp:
            point = point.time(timestamp)

        self.write_api.write(bucket=bucket, record=point)

    def write_sensor_reading(
        self,
        asset_id: str,
        sensor_type: str,
```

```
    values: Dict[str, float],
    timestamp: datetime = None,
    bucket: str = "sensor_data"
):
    """Write sensor reading"""
    self.write_point(
        measurement="sensor_readings",
        tags={"asset_id": asset_id, "sensor_type": sensor_type},
        fields=values,
        timestamp=timestamp,
        bucket=bucket
    )

def write_dataframe(
    self,
    df: pd.DataFrame,
    measurement: str,
    tag_columns: List[str],
    time_column: str = "_time",
    bucket: str = "sensor_data"
):
    """Write DataFrame to InfluxDB"""
    from influxdb_client.client.write_api import WriteOptions

    write_api = self.client.write_api(
        write_options=WriteOptions(
            batch_size=5000,
            flush_interval=10_000
        )
    )

    write_api.write(
        bucket=bucket,
        record=df,
        data_frame_measurement_name=measurement,
        data_frame_tag_columns=tag_columns,
        data_frame_timestamp_column=time_column
    )

    write_api.close()

def write_prediction(
    self,
    asset_id: str,
    model_name: str,
    prediction_type: str,
    predicted_value: float,
    confidence: float,
    timestamp: datetime = None
):
    """Write ML prediction"""
    self.write_point(
        measurement="predictions",
        tags={
```

```
            "asset_id": asset_id,
            "model_name": model_name,
            "prediction_type": prediction_type
        },
        fields={
            "predicted_value": predicted_value,
            "confidence": confidence
        },
        timestamp=timestamp,
        bucket="predictions"
    )

# ========================================
# Downsampling & Retention
# ========================================

def create_downsampling_task(
    self,
    source_bucket: str,
    dest_bucket: str,
    measurement: str,
    interval: str = "1h",
    agg_func: str = "mean"
) -> str:
    """Create downsampling task"""
    task_flux = f'''
    option task = {{
      name: "downsample_{measurement}_{interval}",
      every: {interval}
    }}

    from(bucket: "{source_bucket}")
      |> range(start: -{interval})
      |> filter(fn: (r) => r._measurement == "{measurement}")
      |> aggregateWindow(every: {interval}, fn: {agg_func}, createEmpty: false)
      |> to(bucket: "{dest_bucket}", org: "{self.org}")
    '''

    tasks_api = self.client.tasks_api()
    task = tasks_api.create_task_every(
        name=f"downsample_{measurement}_{interval}",
        flux=task_flux,
        every=interval,
        org=self.org
    )

    return task.id

# ========================================
# Health Check
# ========================================

def health_check(self) -> Dict:
    """Check InfluxDB health"""
```

```
        try:
            health = self.client.health()
            return {
                'status': 'healthy' if health.status == 'pass' else 'unhealthy',
                'version': health.version,
                'message': health.message
            }
        except Exception as e:
            return {
                'status': 'unhealthy',
                'error': str(e)
            }


# CLI usage
if __name__ == '__main__':
    import argparse

    parser = argparse.ArgumentParser(description='InfluxDB Client')
    parser.add_argument('command', choices=['health', 'assets', 'query', 'buckets'])
    parser.add_argument('--asset', help='Asset ID')
    parser.add_argument('--start', default='-24h', help='Start time')
    parser.add_argument('--output', help='Output file')

    args = parser.parse_args()

    with InfluxDBClientWrapper() as client:
        if args.command == 'health':
            print(client.health_check())
        elif args.command == 'assets':
            assets = client.list_assets()
            for asset in assets:
                print(asset)
        elif args.command == 'buckets':
            measurements = client.list_measurements()
            for m in measurements:
                print(m)
        elif args.command == 'query':
            if args.asset:
                df = client.query_sensor_data(args.asset, args.start)
                if args.output:
                    df.to_csv(args.output, index=False)
                else:
                    print(df.to_string())
```

---

4. MILVUS VECTOR DATABASE

4.1 Installation & Setup

```
# Install pymilvus
pip install pymilvus
```

```
# Verify connection
python -c "from pymilvus import connections; connections.connect('default', host='$MILVUS_HOST', port='$
```

4.2 Python Milvus Client

```
#!/usr/bin/env python3
# scripts/milvus_client.py
"""
Milvus Vector Database Client

Usage:
    from milvus_client import MilvusClient
    client = MilvusClient()
    results = client.search_similar("document_embeddings", query_vector, limit=10)
"""

import os
from typing import List, Dict, Any, Optional
import numpy as np
from pymilvus import (
    connections,
    Collection,
    CollectionSchema,
    FieldSchema,
    DataType,
    utility
)
from loguru import logger


class MilvusClient:
    """Milvus vector database operations"""

    def __init__(self):
        self.host = os.environ.get('MILVUS_HOST', 'localhost')
        self.port = os.environ.get('MILVUS_PORT', '19530')
        self.alias = 'default'

        self._connect()

    def _connect(self):
        """Establish connection to Milvus"""
        connections.connect(
            alias=self.alias,
            host=self.host,
            port=self.port
        )
        logger.info(f"Connected to Milvus at {self.host}:{self.port}")

    def close(self):
        """Close connection"""
        connections.disconnect(self.alias)

    # ==========================================
```

```python
    # Collection Operations
    # ========================================

    def list_collections(self) -> List[str]:
        """List all collections"""
        return utility.list_collections()

    def has_collection(self, name: str) -> bool:
        """Check if collection exists"""
        return utility.has_collection(name)

    def create_document_collection(
        self,
        name: str = "document_embeddings",
        dim: int = 1024,
        description: str = "Document embeddings for RAG"
    ):
        """Create collection for document embeddings"""
        if self.has_collection(name):
            logger.warning(f"Collection {name} already exists")
            return

        fields = [
            FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
            FieldSchema(name="document_id", dtype=DataType.VARCHAR, max_length=64),
            FieldSchema(name="chunk_id", dtype=DataType.VARCHAR, max_length=64),
            FieldSchema(name="text", dtype=DataType.VARCHAR, max_length=65535),
            FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=dim),
            FieldSchema(name="metadata", dtype=DataType.JSON)
        ]

        schema = CollectionSchema(fields=fields, description=description)
        collection = Collection(name=name, schema=schema)

        # Create HNSW index for fast search
        index_params = {
            "metric_type": "COSINE",
            "index_type": "HNSW",
            "params": {"M": 16, "efConstruction": 256}
        }

        collection.create_index(field_name="embedding", index_params=index_params)
        logger.info(f"Created collection {name} with HNSW index")

        return collection

    def create_query_cache_collection(
        self,
        name: str = "query_cache",
        dim: int = 1024
    ):
        """Create collection for semantic query cache"""
        if self.has_collection(name):
            logger.warning(f"Collection {name} already exists")
```

```
            return

        fields = [
            FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
            FieldSchema(name="query_text", dtype=DataType.VARCHAR, max_length=2048),
            FieldSchema(name="query_embedding", dtype=DataType.FLOAT_VECTOR, dim=dim),
            FieldSchema(name="response", dtype=DataType.VARCHAR, max_length=65535),
            FieldSchema(name="created_at", dtype=DataType.INT64),
            FieldSchema(name="hits", dtype=DataType.INT64)
        ]

        schema = CollectionSchema(fields=fields, description="Semantic query cache")
        collection = Collection(name=name, schema=schema)

        index_params = {
            "metric_type": "COSINE",
            "index_type": "IVF_FLAT",
            "params": {"nlist": 128}
        }

        collection.create_index(field_name="query_embedding", index_params=index_params)

        return collection

    def drop_collection(self, name: str):
        """Drop collection"""
        if self.has_collection(name):
            utility.drop_collection(name)
            logger.info(f"Dropped collection {name}")

    def get_collection_stats(self, name: str) -> Dict:
        """Get collection statistics"""
        collection = Collection(name)
        collection.load()

        return {
            'name': name,
            'num_entities': collection.num_entities,
            'schema': str(collection.schema),
            'indexes': [idx.to_dict() for idx in collection.indexes]
        }

    # ========================================
    # Insert Operations
    # ========================================

    def insert_document_chunks(
        self,
        collection_name: str,
        document_id: str,
        chunks: List[Dict[str, Any]]
    ) -> List[int]:
        """
        Insert document chunks with embeddings.
```

```
        Args:
            collection_name: Target collection
            document_id: Source document ID
            chunks: List of dicts with keys: chunk_id, text, embedding, metadata
        """
        collection = Collection(collection_name)

        data = {
            'document_id': [document_id] * len(chunks),
            'chunk_id': [c['chunk_id'] for c in chunks],
            'text': [c['text'] for c in chunks],
            'embedding': [c['embedding'] for c in chunks],
            'metadata': [c.get('metadata', {}) for c in chunks]
        }

        result = collection.insert([
            data['document_id'],
            data['chunk_id'],
            data['text'],
            data['embedding'],
            data['metadata']
        ])

        collection.flush()
        logger.info(f"Inserted {len(chunks)} chunks for document {document_id}")

        return result.primary_keys

    def insert_query_cache(
        self,
        query_text: str,
        query_embedding: List[float],
        response: str
    ) -> int:
        """Insert query-response pair into cache"""
        import time

        collection = Collection("query_cache")

        result = collection.insert([
            [query_text],
            [query_embedding],
            [response],
            [int(time.time())],
            [0]
        ])

        collection.flush()
        return result.primary_keys[0]

    # ========================================
    # Search Operations
    # ========================================
```

```
def search_similar(
    self,
    collection_name: str,
    query_embedding: List[float],
    limit: int = 10,
    output_fields: List[str] = None,
    filter_expr: str = None
) -> List[Dict]:
    """
    Search for similar vectors.

    Args:
        collection_name: Collection to search
        query_embedding: Query vector
        limit: Number of results
        output_fields: Fields to return
        filter_expr: Filter expression (e.g., "document_id == 'DOC001'")
    """
    collection = Collection(collection_name)
    collection.load()

    if output_fields is None:
        output_fields = ["document_id", "chunk_id", "text", "metadata"]

    search_params = {
        "metric_type": "COSINE",
        "params": {"ef": 128}
    }

    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param=search_params,
        limit=limit,
        output_fields=output_fields,
        expr=filter_expr
    )

    hits = []
    for hit in results[0]:
        hits.append({
            'id': hit.id,
            'score': hit.score,
            **{field: hit.entity.get(field) for field in output_fields}
        })

    return hits

def semantic_cache_lookup(
    self,
    query_embedding: List[float],
    threshold: float = 0.95
) -> Optional[Dict]:
```

```
        """
        Lookup similar query in cache.

        Args:
            query_embedding: Query vector
            threshold: Similarity threshold (0.95 = 95% similar)
        """
        collection = Collection("query_cache")
        collection.load()

        results = collection.search(
            data=[query_embedding],
            anns_field="query_embedding",
            param={"metric_type": "COSINE", "params": {"nprobe": 10}},
            limit=1,
            output_fields=["query_text", "response", "hits"]
        )

        if results[0] and results[0][0].score >= threshold:
            hit = results[0][0]

            # Update hit count
            self._increment_cache_hits(hit.id)

            return {
                'query_text': hit.entity.get('query_text'),
                'response': hit.entity.get('response'),
                'score': hit.score,
                'cache_hit': True
            }

        return None

    def _increment_cache_hits(self, entity_id: int):
        """Increment cache hit counter"""
        # Note: Milvus doesn't support UPDATE, so we'd need to delete and reinsert
        # In practice, use a separate counter in Redis
        pass

    # ========================================
    # Delete Operations
    # ========================================

    def delete_by_document(self, collection_name: str, document_id: str):
        """Delete all chunks for a document"""
        collection = Collection(collection_name)
        collection.delete(expr=f'document_id == "{document_id}"')
        logger.info(f"Deleted chunks for document {document_id}")

    def delete_by_ids(self, collection_name: str, ids: List[int]):
        """Delete by primary key IDs"""
        collection = Collection(collection_name)
        collection.delete(expr=f'id in {ids}')
```

```python
    # ========================================
    # RAG Pipeline Integration
    # ========================================

    def rag_retrieve(
        self,
        query_embedding: List[float],
        top_k: int = 20,
        rerank_top_k: int = 5,
        document_filter: str = None
    ) -> List[Dict]:
        """
        RAG retrieval pipeline.

        1. Vector search for top_k candidates
        2. Rerank to top rerank_top_k
        3. Return with context
        """
        # Step 1: Vector search
        candidates = self.search_similar(
            "document_embeddings",
            query_embedding,
            limit=top_k,
            filter_expr=document_filter
        )

        # Step 2: Rerank (simple score-based, replace with cross-encoder in production)
        candidates_sorted = sorted(candidates, key=lambda x: x['score'], reverse=True)
        top_results = candidates_sorted[:rerank_top_k]

        # Step 3: Enrich with context
        for result in top_results:
            result['context'] = result.get('text', '')

        return top_results

    # ========================================
    # Health Check
    # ========================================

    def health_check(self) -> Dict:
        """Check Milvus health"""
        try:
            collections = self.list_collections()
            return {
                'status': 'healthy',
                'connected': True,
                'collections': len(collections),
                'collection_names': collections
            }
        except Exception as e:
            return {
                'status': 'unhealthy',
                'connected': False,
```

```
                'error': str(e)
            }


# CLI usage
if __name__ == '__main__':
    import argparse

    parser = argparse.ArgumentParser(description='Milvus Client')
    parser.add_argument('command', choices=['health', 'collections', 'create', 'stats', 'drop'])
    parser.add_argument('--name', help='Collection name')
    parser.add_argument('--dim', type=int, default=1024, help='Vector dimension')

    args = parser.parse_args()

    client = MilvusClient()

    if args.command == 'health':
        print(client.health_check())
    elif args.command == 'collections':
        for name in client.list_collections():
            print(name)
    elif args.command == 'create':
        if args.name:
            client.create_document_collection(args.name, args.dim)
        else:
            client.create_document_collection()
            client.create_query_cache_collection()
    elif args.command == 'stats':
        if args.name:
            print(client.get_collection_stats(args.name))
    elif args.command == 'drop':
        if args.name:
            client.drop_collection(args.name)

    client.close()
```

---

5. REDIS CACHE OPERATIONS

5.1 Installation & Setup

```
# Install Redis CLI
sudo apt-get install -y redis-tools

# Install Python client
pip install redis

# Test connection
redis-cli -h $REDIS_HOST -p $REDIS_PORT -a $REDIS_PASSWORD ping
```

5.2 Redis CLI Commands

```
# Connect to Redis
redis-cli -h $REDIS_HOST -p $REDIS_PORT -a $REDIS_PASSWORD

# Basic operations
SET mykey "Hello"
GET mykey
DEL mykey

# Check if key exists
EXISTS mykey

# Set with expiration (seconds)
SETEX session:user123 3600 "session_data"

# Set with expiration (milliseconds)
PSETEX cache:query:abc 60000 "cached_result"

# Get TTL
TTL session:user123
PTTL cache:query:abc

# Hash operations
HSET user:1001 name "John" email "john@example.com" role "admin"
HGET user:1001 name
HGETALL user:1001
HDEL user:1001 role

# List operations
LPUSH queue:tasks "task1" "task2"
RPOP queue:tasks
LRANGE queue:tasks 0 -1

# Set operations
SADD assets:operational "PUMP-001" "PUMP-002" "COMP-001"
SMEMBERS assets:operational
SISMEMBER assets:operational "PUMP-001"

# Sorted set (scores)
ZADD alerts:priority 1 "alert-001" 5 "alert-002" 10 "alert-003"
ZRANGE alerts:priority 0 -1 WITHSCORES
ZRANGEBYSCORE alerts:priority 5 10

# Key patterns
KEYS session:*
KEYS cache:*

# Delete pattern
redis-cli -h $REDIS_HOST -a $REDIS_PASSWORD KEYS "cache:*" | xargs redis-cli -h $REDIS_HOST -a $REDIS_PA

# Info
INFO memory
INFO clients
```

```
INFO stats

# Database size
DBSIZE

# Flush database (careful!)
# FLUSHDB
# FLUSHALL
```

5.3 Python Redis Client

```
#!/usr/bin/env python3
# scripts/redis_client.py
"""
Redis Cache Client

Usage:
    from redis_client import RedisClient
    cache = RedisClient()
    cache.set_json("key", {"data": "value"}, ttl=3600)
"""

import os
import json
from typing import Any, Dict, List, Optional
from datetime import timedelta
import redis
from loguru import logger


class RedisClient:
    """Redis cache operations"""

    def __init__(self):
        self.host = os.environ.get('REDIS_HOST', 'localhost')
        self.port = int(os.environ.get('REDIS_PORT', 6379))
        self.password = os.environ.get('REDIS_PASSWORD')
        self.db = int(os.environ.get('REDIS_DB', 0))

        self.client = redis.Redis(
            host=self.host,
            port=self.port,
            password=self.password,
            db=self.db,
            decode_responses=True
        )

    # ========================================
    # Basic Operations
    # ========================================

    def get(self, key: str) -> Optional[str]:
        """Get string value"""
        return self.client.get(key)
```

```
    def set(self, key: str, value: str, ttl: int = None):
        """Set string value with optional TTL (seconds)"""
        if ttl:
            self.client.setex(key, ttl, value)
        else:
            self.client.set(key, value)

    def delete(self, key: str) -> bool:
        """Delete key"""
        return self.client.delete(key) > 0

    def exists(self, key: str) -> bool:
        """Check if key exists"""
        return self.client.exists(key) > 0

    def ttl(self, key: str) -> int:
        """Get TTL in seconds (-1 = no expire, -2 = not exists)"""
        return self.client.ttl(key)

    def expire(self, key: str, seconds: int) -> bool:
        """Set expiration"""
        return self.client.expire(key, seconds)

    # ========================================
    # JSON Operations
    # ========================================

    def get_json(self, key: str) -> Optional[Dict]:
        """Get JSON value"""
        value = self.client.get(key)
        if value:
            return json.loads(value)
        return None

    def set_json(self, key: str, value: Dict, ttl: int = None):
        """Set JSON value"""
        self.set(key, json.dumps(value), ttl)

    # ========================================
    # Hash Operations (for objects)
    # ========================================

    def hget(self, key: str, field: str) -> Optional[str]:
        """Get hash field"""
        return self.client.hget(key, field)

    def hset(self, key: str, field: str, value: str):
        """Set hash field"""
        self.client.hset(key, field, value)

    def hgetall(self, key: str) -> Dict[str, str]:
        """Get all hash fields"""
        return self.client.hgetall(key)
```

```python
def hmset(self, key: str, mapping: Dict):
    """Set multiple hash fields"""
    self.client.hset(key, mapping=mapping)

def hdel(self, key: str, *fields):
    """Delete hash fields"""
    self.client.hdel(key, *fields)

# ========================================
# Session Management
# ========================================

def create_session(self, user_id: str, session_data: Dict, ttl: int = 3600) -> str:
    """Create user session"""
    import uuid
    session_id = str(uuid.uuid4())
    key = f"session:{session_id}"

    self.set_json(key, {
        'user_id': user_id,
        **session_data
    }, ttl)

    # Track user sessions
    self.client.sadd(f"user_sessions:{user_id}", session_id)

    return session_id

def get_session(self, session_id: str) -> Optional[Dict]:
    """Get session data"""
    return self.get_json(f"session:{session_id}")

def delete_session(self, session_id: str):
    """Delete session"""
    session = self.get_session(session_id)
    if session:
        user_id = session.get('user_id')
        self.delete(f"session:{session_id}")
        if user_id:
            self.client.srem(f"user_sessions:{user_id}", session_id)

def get_user_sessions(self, user_id: str) -> List[str]:
    """Get all sessions for a user"""
    return list(self.client.smembers(f"user_sessions:{user_id}"))

# ========================================
# Query Cache
# ========================================

def cache_query(
    self,
    query_key: str,
    result: Any,
```

```
    ttl: int = 300
):
    """Cache query result"""
    key = f"cache:query:{query_key}"
    self.set_json(key, result, ttl)

def get_cached_query(self, query_key: str) -> Optional[Any]:
    """Get cached query result"""
    return self.get_json(f"cache:query:{query_key}")

def invalidate_query_cache(self, pattern: str = "*"):
    """Invalidate query cache by pattern"""
    keys = self.client.keys(f"cache:query:{pattern}")
    if keys:
        self.client.delete(*keys)
        logger.info(f"Invalidated {len(keys)} cache entries")

# ========================================
# Rate Limiting
# ========================================

def rate_limit_check(
    self,
    key: str,
    limit: int,
    window: int = 60
) -> Dict:
    """
    Check rate limit using sliding window.

    Args:
        key: Identifier (e.g., user_id, IP)
        limit: Maximum requests per window
        window: Window size in seconds

    Returns:
        Dict with 'allowed', 'remaining', 'reset_at'
    """
    import time

    now = time.time()
    window_start = now - window
    bucket_key = f"ratelimit:{key}"

    # Remove old entries
    self.client.zremrangebyscore(bucket_key, 0, window_start)

    # Count current requests
    current_count = self.client.zcard(bucket_key)

    if current_count < limit:
        # Add new request
        self.client.zadd(bucket_key, {str(now): now})
        self.client.expire(bucket_key, window)
```

```
            return {
                'allowed': True,
                'remaining': limit - current_count - 1,
                'reset_at': int(now + window)
            }
        else:
            # Get oldest entry for reset time
            oldest = self.client.zrange(bucket_key, 0, 0, withscores=True)
            reset_at = int(oldest[0][1] + window) if oldest else int(now + window)

            return {
                'allowed': False,
                'remaining': 0,
                'reset_at': reset_at
            }

    # ========================================
    # Pub/Sub
    # ========================================

    def publish(self, channel: str, message: Any):
        """Publish message to channel"""
        if isinstance(message, dict):
            message = json.dumps(message)
        self.client.publish(channel, message)

    def subscribe(self, channels: List[str]):
        """Subscribe to channels"""
        pubsub = self.client.pubsub()
        pubsub.subscribe(*channels)
        return pubsub

    # ========================================
    # Health Check
    # ========================================

    def health_check(self) -> Dict:
        """Check Redis health"""
        try:
            self.client.ping()
            info = self.client.info('memory')

            return {
                'status': 'healthy',
                'connected': True,
                'used_memory': info.get('used_memory_human'),
                'connected_clients': self.client.info('clients').get('connected_clients')
            }
        except Exception as e:
            return {
                'status': 'unhealthy',
                'connected': False,
                'error': str(e)
```

```
            }


# CLI usage
if __name__ == '__main__':
    import argparse

    parser = argparse.ArgumentParser(description='Redis Client')
    parser.add_argument('command', choices=['health', 'get', 'set', 'del', 'keys', 'flush-cache'])
    parser.add_argument('--key', help='Key name')
    parser.add_argument('--value', help='Value')
    parser.add_argument('--ttl', type=int, help='TTL in seconds')
    parser.add_argument('--pattern', default='*', help='Key pattern')

    args = parser.parse_args()

    client = RedisClient()

    if args.command == 'health':
        print(client.health_check())
    elif args.command == 'get':
        print(client.get(args.key))
    elif args.command == 'set':
        client.set(args.key, args.value, args.ttl)
        print(f"Set {args.key}")
    elif args.command == 'del':
        client.delete(args.key)
        print(f"Deleted {args.key}")
    elif args.command == 'keys':
        keys = client.client.keys(args.pattern)
        for key in keys:
            print(key)
    elif args.command == 'flush-cache':
        client.invalidate_query_cache(args.pattern)
```

---

6. DATABASE MIGRATION TOOLS

6.1 Alembic (PostgreSQL Migrations)

```
# Install Alembic
pip install alembic

# Initialize Alembic
cd /path/to/project
alembic init alembic

# Configure alembic.ini
# sqlalchemy.url = postgresql://user:pass@host/db

# Create migration
alembic revision --autogenerate -m "Add sensors table"
```

```
# Apply migrations
alembic upgrade head

# Rollback
alembic downgrade -1

# Show current version
alembic current

# Show history
alembic history
```

6.2 Migration Script Example

```
# alembic/versions/001_initial_schema.py
"""Initial schema

Revision ID: 001
Create Date: 2025-12-10
"""
from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = '001'
down_revision = None
branch_labels = None
depends_on = None


def upgrade():
    # Create asset_mgmt schema
    op.execute('CREATE SCHEMA IF NOT EXISTS asset_mgmt')

    # Create assets table
    op.create_table(
        'assets',
        sa.Column('asset_id', sa.Integer(), primary_key=True),
        sa.Column('asset_tag', sa.String(50), unique=True, nullable=False),
        sa.Column('name', sa.String(200), nullable=False),
        sa.Column('asset_type', sa.String(50), nullable=False),
        sa.Column('status', sa.String(20), default='operational'),
        sa.Column('criticality', sa.String(20), default='medium'),
        sa.Column('location_id', sa.Integer()),
        sa.Column('metadata', postgresql.JSONB()),
        sa.Column('created_at', sa.DateTime(), server_default=sa.func.now()),
        sa.Column('updated_at', sa.DateTime(), server_default=sa.func.now()),
        schema='asset_mgmt'
    )

    # Create indexes
    op.create_index(
        'idx_assets_asset_tag',
        'assets',
```

```
        ['asset_tag'],
        schema='asset_mgmt'
    )

    op.create_index(
        'idx_assets_status',
        'assets',
        ['status'],
        schema='asset_mgmt'
    )


def downgrade():
    op.drop_table('assets', schema='asset_mgmt')
    op.execute('DROP SCHEMA IF EXISTS asset_mgmt CASCADE')
```

---

7. QUERY OPTIMIZATION

7.1 PostgreSQL Query Optimization

```
-- Analyze query plan
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT * FROM asset_mgmt.assets WHERE status = 'operational';

-- Check index usage
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
WHERE schemaname = 'asset_mgmt'
ORDER BY idx_scan DESC;

-- Find missing indexes
SELECT
    relname as table,
    seq_scan,
    seq_tup_read,
    idx_scan,
    idx_tup_fetch,
    seq_tup_read / NULLIF(seq_scan, 0) as avg_seq_tup_read
FROM pg_stat_user_tables
WHERE seq_scan > 100
AND seq_tup_read / NULLIF(seq_scan, 0) > 1000
ORDER BY seq_tup_read DESC;

-- Create missing indexes
CREATE INDEX CONCURRENTLY idx_sensors_asset_id
ON asset_mgmt.sensors(asset_id);
```

```sql
-- Analyze tables for better query planning
ANALYZE asset_mgmt.assets;
ANALYZE asset_mgmt.sensors;

-- Vacuum to reclaim space
VACUUM (VERBOSE, ANALYZE) asset_mgmt.assets;
```

7.2 InfluxDB Query Optimization

```
# Use specific time ranges
influx query '
from(bucket: "sensor_data")
  |> range(start: -1h)  // Better than start: 0
  |> filter(fn: (r) => r._measurement == "sensor_readings")
'

# Push down filters
influx query '
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) =>
      r._measurement == "sensor_readings" and
      r.asset_id == "PUMP-001" and
      r._field == "vibration"
  )
  // Filters before aggregation
  |> aggregateWindow(every: 5m, fn: mean)
'

# Use yield for multiple outputs
influx query '
data = from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._field == "vibration")

data |> mean() |> yield(name: "mean")
data |> max() |> yield(name: "max")
data |> min() |> yield(name: "min")
'
```

---

8. BACKUP & RECOVERY

8.1 Backup Scripts

```
#!/bin/bash
# scripts/backup_databases.sh
# Comprehensive database backup

set -e

BACKUP_DIR="${1:-/backup/$(date +%Y%m%d_%H%M%S)}"
```

```
mkdir -p "$BACKUP_DIR"

echo "Database Backup - $(date)"
echo "=========================="

# PostgreSQL
echo "Backing up PostgreSQL..."
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -Fc --no-owner -f "$BACKUP_DIR/postgresql.dump"
echo "  Size: $(du -h $BACKUP_DIR/postgresql.dump | cut -f1)"

# InfluxDB
echo "Backing up InfluxDB..."
influx backup "$BACKUP_DIR/influxdb" \
    --host $INFLUXDB_URL \
    --token $INFLUXDB_TOKEN
echo "  Size: $(du -sh $BACKUP_DIR/influxdb | cut -f1)"

# Redis (RDB snapshot)
echo "Backing up Redis..."
redis-cli -h $REDIS_HOST -a $REDIS_PASSWORD BGSAVE
sleep 5  # Wait for save
redis-cli -h $REDIS_HOST -a $REDIS_PASSWORD --rdb "$BACKUP_DIR/redis.rdb"
echo "  Size: $(du -h $BACKUP_DIR/redis.rdb | cut -f1)"

# Milvus (metadata only - vectors should be re-indexed)
echo "Backing up Milvus metadata..."
python3 << EOF
from pymilvus import connections, utility, Collection
import json

connections.connect(host="${MILVUS_HOST}", port="${MILVUS_PORT}")

backup = {
    'collections': []
}

for name in utility.list_collections():
    coll = Collection(name)
    backup['collections'].append({
        'name': name,
        'schema': str(coll.schema),
        'num_entities': coll.num_entities
    })

with open('$BACKUP_DIR/milvus_metadata.json', 'w') as f:
    json.dump(backup, f, indent=2)
EOF

# Create manifest
cat > "$BACKUP_DIR/manifest.json" << EOF
{
  "backup_time": "$(date -Iseconds)",
  "databases": {
```

```
    "postgresql": "postgresql.dump",
    "influxdb": "influxdb/",
    "redis": "redis.rdb",
    "milvus": "milvus_metadata.json"
  }
}
EOF

# Compress
tar czf "${BACKUP_DIR}.tar.gz" -C "$(dirname $BACKUP_DIR)" "$(basename $BACKUP_DIR)"
rm -rf "$BACKUP_DIR"

echo ""
echo "Backup complete: ${BACKUP_DIR}.tar.gz"
echo "Total size: $(du -h ${BACKUP_DIR}.tar.gz | cut -f1)"
```

8.2 Restore Scripts

```
#!/bin/bash
# scripts/restore_databases.sh
# Restore databases from backup

set -e

BACKUP_FILE="$1"

if [ -z "$BACKUP_FILE" ]; then
    echo "Usage: $0 <backup_file.tar.gz>"
    exit 1
fi

RESTORE_DIR="/tmp/restore_$(date +%s)"
mkdir -p "$RESTORE_DIR"

echo "Extracting backup..."
tar xzf "$BACKUP_FILE" -C "$RESTORE_DIR"
BACKUP_DIR=$(ls "$RESTORE_DIR")

# PostgreSQL
echo "Restoring PostgreSQL..."
pg_restore -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -c --if-exists "$RESTORE_DIR/$BACKUP_DIR/postgresql.dump"

# InfluxDB
echo "Restoring InfluxDB..."
influx restore "$RESTORE_DIR/$BACKUP_DIR/influxdb" \
    --host $INFLUXDB_URL \
    --token $INFLUXDB_TOKEN \
    --full

# Redis
echo "Restoring Redis..."
redis-cli -h $REDIS_HOST -a $REDIS_PASSWORD FLUSHALL
redis-cli -h $REDIS_HOST -a $REDIS_PASSWORD --rdb "$RESTORE_DIR/$BACKUP_DIR/redis.rdb"
```

```
# Cleanup
rm -rf "$RESTORE_DIR"

echo "Restore complete!"
```

---

9. MONITORING & HEALTH CHECKS

9.1 Health Check Script

```
#!/usr/bin/env python3
# scripts/database_health_check.py
"""
Database Health Check

Usage:
    python database_health_check.py
    python database_health_check.py --json
"""

import argparse
import json
from postgres_client import PostgresClient
from influxdb_client_wrapper import InfluxDBClientWrapper
from milvus_client import MilvusClient
from redis_client import RedisClient


def check_all_databases() -> dict:
    """Check health of all databases"""
    results = {}

    # PostgreSQL
    try:
        pg = PostgresClient()
        results['postgresql'] = pg.health_check()
    except Exception as e:
        results['postgresql'] = {'status': 'error', 'error': str(e)}

    # InfluxDB
    try:
        with InfluxDBClientWrapper() as influx:
            results['influxdb'] = influx.health_check()
    except Exception as e:
        results['influxdb'] = {'status': 'error', 'error': str(e)}

    # Milvus
    try:
        milvus = MilvusClient()
        results['milvus'] = milvus.health_check()
        milvus.close()
    except Exception as e:
```

```
        results['milvus'] = {'status': 'error', 'error': str(e)}

    # Redis
    try:
        redis = RedisClient()
        results['redis'] = redis.health_check()
    except Exception as e:
        results['redis'] = {'status': 'error', 'error': str(e)}

    # Overall status
    all_healthy = all(
        r.get('status') == 'healthy'
        for r in results.values()
    )
    results['overall'] = 'healthy' if all_healthy else 'degraded'

    return results


def main():
    parser = argparse.ArgumentParser(description='Database Health Check')
    parser.add_argument('--json', action='store_true', help='Output as JSON')

    args = parser.parse_args()

    results = check_all_databases()

    if args.json:
        print(json.dumps(results, indent=2))
    else:
        print("Database Health Check")
        print("=" * 40)

        for db, status in results.items():
            if db == 'overall':
                continue

            status_icon = 'OK' if status.get('status') == 'healthy' else 'FAIL'
            print(f"{db}: [{status_icon}]")

            if status.get('error'):
                print(f"  Error: {status['error']}")
            else:
                for key, value in status.items():
                    if key != 'status':
                        print(f"  {key}: {value}")

        print("=" * 40)
        print(f"Overall: {results['overall'].upper()}")


if __name__ == '__main__':
    main()
```

10. COMPLETE OPERATION WORKFLOWS

10.1 New Asset Onboarding

```
#!/bin/bash
# workflows/onboard_asset.sh
# Complete workflow for onboarding a new asset

ASSET_TAG="$1"
ASSET_NAME="$2"
ASSET_TYPE="${3:-pump}"

if [ -z "$ASSET_TAG" ] || [ -z "$ASSET_NAME" ]; then
    echo "Usage: $0 <asset_tag> <asset_name> [asset_type]"
    exit 1
fi

echo "Onboarding Asset: $ASSET_TAG"
echo "=============================="

# 1. Create asset in PostgreSQL
echo "Step 1: Creating asset record..."
psql -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB << EOF
INSERT INTO asset_mgmt.assets (asset_tag, name, asset_type, status, criticality)
VALUES ('$ASSET_TAG', '$ASSET_NAME', '$ASSET_TYPE', 'commissioning', 'medium')
ON CONFLICT (asset_tag) DO UPDATE SET status = 'commissioning', updated_at = NOW()
RETURNING asset_id;
EOF

# 2. Create InfluxDB measurements
echo "Step 2: Initializing InfluxDB..."
influx write --bucket sensor_data \
    "sensor_readings,asset_id=$ASSET_TAG,sensor_type=init value=0"

# 3. Initialize vector collection if needed
echo "Step 3: Checking Milvus..."
python3 -c "
from milvus_client import MilvusClient
client = MilvusClient()
if not client.has_collection('document_embeddings'):
    client.create_document_collection()
client.close()
"

# 4. Create Redis cache entries
echo "Step 4: Initializing cache..."
redis-cli -h $REDIS_HOST -a $REDIS_PASSWORD HSET "asset:$ASSET_TAG" status "commissioning" type "$ASSET

echo ""
echo "Asset $ASSET_TAG onboarded successfully!"
```

10.2 Quick Reference

```
# ============================================
# DATABASE QUICK REFERENCE
# ============================================

# --- POSTGRESQL ---
psql -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB
\dt asset_mgmt.*                    # List tables
\d+ asset_mgmt.assets               # Describe table
SELECT * FROM asset_mgmt.assets LIMIT 10;

# --- INFLUXDB ---
influx query 'from(bucket:"sensor_data") |> range(start:-1h) |> limit(n:10)'
influx bucket list
influx write --bucket sensor_data 'test,tag=value field=1.0'

# --- MILVUS ---
python -c "from milvus_client import MilvusClient; c=MilvusClient(); print(c.list_collections())"

# --- REDIS ---
redis-cli -h $REDIS_HOST -a $REDIS_PASSWORD
KEYS *                              # List all keys
GET mykey                           # Get value
INFO memory                         # Memory stats

# --- HEALTH CHECK ---
python scripts/database_health_check.py

# --- BACKUP ---
./scripts/backup_databases.sh /backup/

# --- RESTORE ---
./scripts/restore_databases.sh /backup/backup.tar.gz
```

END OF DATABASE TOOLS & OPERATIONS GUIDE